

ROS Component Explorer

An Intelligent Search & Discovery System for Robotics

Presented by: Riddhesh P More

Advisor: Dr Björn Kahl

Date 12th September 2025



Project Context: IAM4RAIL

"Intelligent Asset Management for RAIL"

Project Goal

To develop a holistic, rail-centric asset management system using AI, data analytics, and robotics to improve the reliability, capacity, and safety of European railway networks.

Our Contribution

The **ROS Component Explorer** is a foundational tool within this vision. It accelerates the development of complex robotic systems for inspection and maintenance by enabling developers to rapidly discover and reuse the right software components.

The Core Problem: The "Semantic Gap"

In ROS, it's easy to connect components, but hard to know if they **should** be connected.

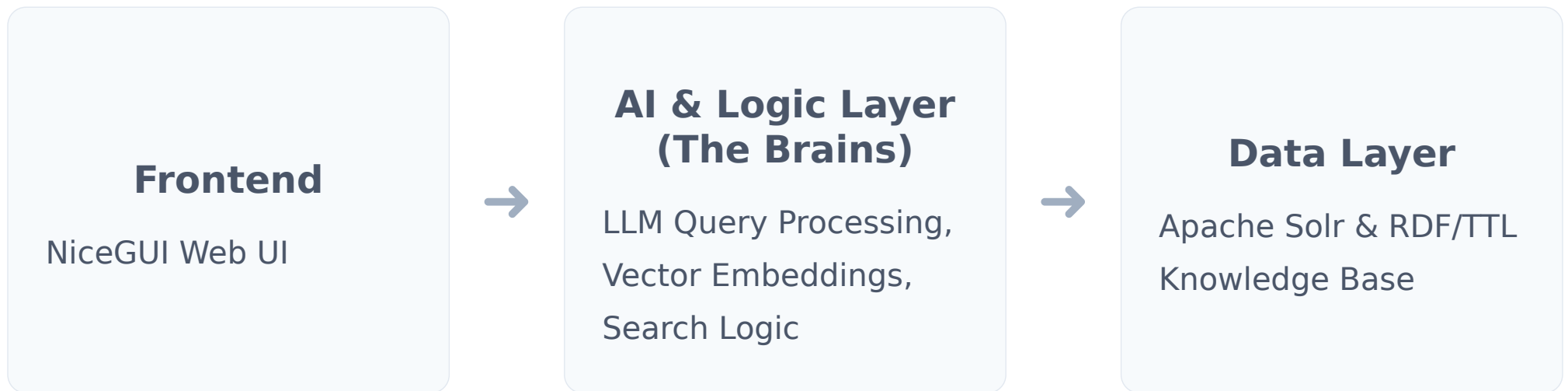
Challenge

Developers rely on keyword searches, which often fail to capture the true functionality, hardware requirements, or compatibility of a ROS package.

Solution

Build an intelligent system that understands queries based on their **meaning**, not just keywords, to provide accurate, context-aware component recommendations.

High-Level Architecture



This layered approach separates user interaction from the complex logic of understanding queries and retrieving data from the specialized knowledge base.

Building the Foundation: The Ontology Model

Before we can store facts, we need a blueprint. This is the Ontology, defined in files like **ros_package_ontology.ttl**. It acts as a formal vocabulary or schema for our knowledge graph.

1. Defining Classes (The "Nouns")

We created a hierarchy of component types. This allows us to understand that a ``LocalizationPackage`` is a type of ``AlgorithmPackage``.

```
# A LocalizationPackage is a type of
AlgorithmPackage
ros:LocalizationPackage
rdfs:subClassOf ros:AlgorithmPackage .
```

2. Defining Properties (The "Verbs")

We defined the relationships components can have, such as dependencies, hardware requirements, and compatibility.

```
# A property to link a package to hardware
ros:requiresHardware a owl:ObjectProperty .
```

This ontology provides the essential **structure** for our knowledge, ensuring data is consistent and queryable.

Populating the Knowledge Graph with Data

With the ontology as our blueprint, we populate it with facts about real ROS packages. This process turns documentation into structured, machine-readable knowledge.

From Fact to Triple

Each piece of information is stored as a "triple" (Subject-Predicate-Object). This is the core of our model.

# Subject	Predicate	Object
#-----	-----	-----
# The 'amcl' package	is a type of	'LocalizationPackage'.
ros:amcl	rdf:type	ros:LocalizationPackage ;
# It has the label		"AMCL".
	rdfs:label	"AMCL" ;
# It requires the hardware		'LaserScanner'.
	ros:requiresHardware	hw:LaserScanner ;
# It is compatible with		the 'move_base' package.
	ros:compatibleWith	ros:move_base .

This structured data is then loaded into Apache Solr, making it a searchable knowledge base.

From Fact to Triple: A Deeper Look

This process is the bedrock of our knowledge model. We are translating human-readable facts into a language that databases and algorithms can understand and reason with.

Subject

The 'thing' we are describing. The central node in a given fact.

```
ros:amcl
```

Predicate

The property of the subject, or the relationship it has.

```
rdf:type  
rdfs:label  
ros:requiresHardware
```

Object

The value of the property, or the other 'thing' it's related to.

```
ros:LocalizationPackage  
"AMCL"  
hw:LaserScanner
```

By repeating this for thousands of facts, we build a complex web, or **graph**, of interconnected knowledge that the system can navigate to answer questions.

Adding "Understanding": Vector Embeddings

The TTL model is for structured facts but doesn't understand natural language. To bridge this gap, we use Vector Embeddings.

Think of it as giving each component a unique "meaning fingerprint" in the form of numbers.

The Process (`vector_generator.py`)

1. Create a rich text description for each component (name, description, topics).
2. Use a Sentence Transformer model (**all-MiniLM-L6-v2**) to convert this text into a 384-dimension vector.
3. Store this vector in Apache Solr alongside the component's other data.

The Result

We can now find components that are **semantically similar**, not just those that share keywords. A search for "path planning" can now find components described as "motion control" or "trajectory generation".

The Query Lifecycle: From You to an Answer

1. User Query: "I need a SLAM package for a robot with a LiDAR."



2. LLM Processing (query_processor.py): The AI identifies key requirements: Category: `SLAM`, Sensor: `LiDAR`.



3. Hybrid Search (llm_search_engine.py):

- **Text Search:** Solr finds documents with "SLAM", "LiDAR".
- **Vector Search:** Solr finds documents with a similar "meaning fingerprint".
- **TTL Enrichment:** Results are enhanced with compatibility info from the graph.



4. AI Synthesis: The LLM combines results and context into a human-readable recommendation.

Conclusion: A Multi-Layered Knowledge Model

The system's intelligence comes from integrating different models, each with a specific strength.

The Ontology (TTL)

Provides the **structured, factual backbone**. It knows **what** things are and how they are formally related.

Vector Embeddings

Adds a layer of **semantic understanding**. It knows what things **mean** and what concepts are similar.

The LLM

Acts as the **intelligent orchestrator**, translating human language and synthesizing results.

This creates a search experience that is both powerful and intuitive, bridging the gap between human intent and machine-readable data.



Future Scope & Research Directions

Inspired by recent academic research, we can enhance the system's intelligence and automation significantly.

1. Automated Knowledge Graph Construction

- Automatically parse ROS repositories (code, `package.xml`, READMEs) to populate the graph.
- Employ **LLM-based autonomous agents** to read documentation and generate semantic triples directly.

2. Domain-Specific Semantic Models

- **Fine-tune a language model** (e.g., BERT-ROS) on a large corpus of ROS documentation for more accurate semantic matching.

3. Advanced Feature Fusion & Ranking

- Replace ad-hoc scoring with a **trained regression model** to intelligently weigh features (Function, Hardware) based on query context.

4. Graph-Native Search & Reasoning

- Implement true graph-native queries to find components based on complex relationship patterns, not just attributes.

Thank You

Questions?